Exception Handling in C#

Practical exercises: throwing and catching exceptions

Introduction

An exception is an error that occurs while a program is running. When it is not handled, the program stops. C# provides three keywords to deal with this: **throw** creates an exception when a rule is broken, **try** wraps the code that might fail, and **catch** receives the exception and decides what to do with it. The two exercises below use **throw** to enforce a rule and **try/catch** to report the problem without crashing.

Exercise 1: Duplicate numbers

Task: Write a C# program that reads a list of integers from the user and throws an exception if any numbers are duplicates.

Solution

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        List<int> numbers = new List<int>();

        Console.Write("How many numbers do you want to enter? ");
        int count = int.Parse(Console.ReadLine());

        try
        {
            for (int i = 0; i < count; i++)
            {
                Console.Write("Enter number " + (i + 1) + ": ");
                int num = int.Parse(Console.ReadLine());

                // If the number is already in the list, throw an exception
                if (numbers.Contains(num))
                {
                    throw new Exception("Duplicate number found: " + num);
                }

                numbers.Add(num);
            }

            Console.WriteLine("No duplicates found.");
        }
        catch (FormatException)
        {
            Console.WriteLine("Error: please enter numbers only.");
        }
        catch (Exception ex)
        {
            Console.WriteLine("Error: " + ex.Message);
        }
    }
}
```

Explanation

- The numbers are stored in a `List<int>`.
- For each new number, `numbers.Contains(num)` checks whether it was already entered.
- If it was, `throw new Exception(...)` stops the loop and moves control to the matching `catch` block.
- `FormatException` is raised by `int.Parse` when the input is not a number, so it has its own `catch`.
- The general `catch (Exception ex)` handles the duplicate-number exception and prints its message.

Sample output

```
How many numbers do you want to enter? 4
Enter number 1: 5
Enter number 2: 8
Enter number 3: 2
Enter number 4: 8
Error: Duplicate number found: 8
```

Exercise 2: String without vowels

Task: Write a C# program to create a method that takes a string as input and throws an exception if the string does not contain vowels.

Solution

```
using System;

class Program
{
    // Throws an exception if the string does not contain any vowels
    static void CheckVowels(string text)
    {
        string vowels = "aeiou";
        bool found = false;

        foreach (char c in text.ToLower())
        {
            if (vowels.Contains(c))
            {
                found = true;
                break; // one vowel is enough, stop searching
            }
        }

        if (!found)
        {
            throw new Exception("The string does not contain any vowels.");
        }
    }

    static void Main()
    {
        Console.Write("Enter a string: ");
        string input = Console.ReadLine();

        try
        {
            CheckVowels(input);
            Console.WriteLine("The string contains vowels.");
        }
        catch (Exception ex)
        {
            Console.WriteLine("Error: " + ex.Message);
        }
    }
}
```

Explanation

- `text.ToLower()` makes the check case-insensitive, so 'A' and 'a' are treated the same.
- The loop checks each character against the string `"aeiou"`.
- When a vowel is found, `found` is set to true and `break` ends the loop early.
- After the loop, if `found` is still false, the method throws an exception.
- `Main` calls the method inside `try` and prints the error message from `catch`.

Sample output

```
Enter a string: hello
The string contains vowels.

Enter a string: rhythm
Error: The string does not contain any vowels.
```

Conclusion

Both exercises validate the input, throw an exception when the rule is broken, and catch it to show a clear message. More specific types such as `ArgumentException` can be used to improve this.